

```

def min_spanning_tree(self, start):
    """
    Finds the minimum spanning tree starting at vertex start using Prim's
    algorithm.
    :return: the list of edges in the minimum spanning tree and the cost of
    it. Throws an error if the starting vertex does not exist.
    """
    if not self.valid_vertex(start):
        raise ValueError("Vertex {} does not exist.".format(start))
    q = [] #priority queue: list of tuples (cost, node)
    prev = {} #stores the parent of each node
    dist = {} #stores the minimum cost to reach each node
    edges = [] #edges in the mst
    total_cost = 0 #total cost of the mst
    vertices = {start} #nodes in the mst

    #initialize distances for neighbors of start
    for x in self.parse_out(start):
        dist[x] = self.get_cost(start, x) #distance up to node x is the cost
of the edge
        prev[x] = start #prev of node x is start
        heapq.heappush(q, (dist[x], x)) #nodes to visit next prioritized by
cost

    for x in self.parse_in(start):
        dist[x] = self.get_cost(x, start)
        prev[x] = start
        heapq.heappush(q, (dist[x], x))

    #build mst
    while q: #repeat while there are still
        cost_x, x = heapq.heappop(q) #pop the node with the smallest cost
        if x not in vertices: #if the node is not already in the mst
            edges.append((prev[x], x)) #add the edge to the mst
            total_cost += cost_x #increase the total cost
            vertices.add(x) #mark node as visited
            for y in self.parse_out(x): #get all the neighbours of node that
was just added to the mst
                cost_xy = self.get_cost(x, y)
                if y not in dist or cost_xy < dist[y]: #if the neighbour is
not visited yet or we found a cheaper edge to get to it
                    dist[y] = cost_xy #the distance is updated
                    prev[y] = x #the prev of y is x
                    heapq.heappush(q, (dist[y], y)) #add it to the queue
            for y in self.parse_in(x):
                cost_xy = self.get_cost(y, x)
                if y not in dist or cost_xy < dist[y]:
                    dist[y] = cost_xy
                    prev[y] = x
                    heapq.heappush(q, (dist[y], y))
    return edges, total_cost

```